

Exercise 1.3: Custom Classes

Lab Objectives

- Learn how to create your own custom Classes
- Design, implement, use and test simple Java classes
- Follow standard Java conventions for class design and implementation
- Write and generate Javadoc class documentation.
- Appreciate the role of UML notation for specifying classes

Associated Reading:

Java Oracle Tutorials,

<https://docs.oracle.com/javase/tutorial/java/javaOO/classes.html>

<https://docs.oracle.com/javase/tutorial/java/javaOO/returnvalue.html>

<https://docs.oracle.com/javase/tutorial/java/javaOO/accesscontrol.html>

<https://docs.oracle.com/javase/tutorial/java/javaOO/summaryclasses.html>

Introduction

Up until now you have been using existing predefined classes from the Java Standard Edition library (e.g. Scanner, String). To start to make your own software applications within Java, rather than writing simple demonstration programs that use existing classes, you will need to start to design and implement your own classes.

You can therefore combine some of the existing tools available to you within the Java library to make your own unique Classes, demonstrate their behaviour, and test them. As you progress throughout the module you will start to build software applications where you will write several custom classes that all communicate to perform given tasks

Class design - standard conventions

A class encapsulates data that is stored within fields, (aka "instance variables"). The class specifies and implements methods that can modify and access the fields, as well as carry out other tasks. There are also constructor methods to create instances of the class, i.e. objects.

Following standard conventions makes your code more understandable and maintainable by others and yourself. Use the following guidelines as a check list:

- ✓ The class name begins with a capital letter. All other identifiers begin with a lowercase letter.
- ✓ Each class is normally coded in a file of the same name.
- ✓ The data fields are declared with a **private** modifier so they are not visible outside the class.
- ✓ Methods that can be accessed by other objects are declared as **public**. (The public interface.)

- ✓ Standard modifier methods begin with the word 'set', e.g. **setCount()**
- ✓ Standard accessor methods begin with the word 'get', e.g. **getCount()**
 - or 'is' if the property is a boolean e.g. **isEmpty()**, **isActive()**
- ✓ The **toString()** method is inherited by all classes, and is usually overridden to give a textual description of the object's state. E.g.
 - **public String toString() { return "Counter:[count=" + count + "]; }**
- ✓ The **equals(...)** method is inherited by all classes, and is usually overridden to allow object instances of that class type to be compared for equality.
- ✓ A custom class does not usually have a **main()** method. It cannot be 'run' on its own. A separate demonstration application can be used for this.
- ✓ Avoid user I/O in a data model class, e.g. **no** **System.out.println** statements, and no **Scanner** **next()** methods from the keyboard or dialogue boxes

Building your own custom Java classes

This is a really important building block and it is fundamental to your progression during this module that you understand how to create classes. You should practice until it becomes second nature; the more different classes you create the better. A basic design and implementation is all that is required for these exercises. We shall improve and elaborate on this during the module.

The objective is to write simple classes of the form:

```
class Example {  
  
    //field declarations (i.e. instance variables)  
  
    // constructor(s)  
  
    //methods  
}
```

We may then have a separate demonstration application (with a main method) to create and use one or more object instances of the class. We may also write unit tests to test each constructor and method of the class.

Question 1 Reopen your Eclipse workspace called "CTEC2710_OO" and then download the existing Java Project required for this lab by extracting it from the "MyFirstClasses.zip" file, and importing the project into Eclipse. You should use this project for the remainder of this lab.

- a) Unveil the contents of the src folder - you will see two packages: lib and main. Firstly take a look at the custom "**Counter**" class within the lib package and study the code and comments. You will notice most of the comments are highlighted in blue - that is because they are Javadoc comments, which are explained in more detail in the parallel **JavaDoc Notes** document you will find on LearningZone.

- b) Now take a look in the main package at the "**CounterApp**" program. You will note the main method as this is a simple Java application that demonstrates some of the Counter's behaviour. Run the file to see what it does and study the code and comments. **Note:** You may ignore the "Errors exist in this project" message, as this is stemming from a test program for a class that does not yet exist, and is therefore currently expected.
- c) To get the blast off code working, you will need to add a decrement() method into the Counter class that should simply reduce the count value by one (i.e. the opposite of increment). Once you have done this, uncomment the code towards the bottom of the CounterApp program and run it again:
- d) Whilst a main program can be used to informally test your classes, a more professional testing tool such as [JUnit](#) is a better solution. You will notice that the MyFirstClasses project has another source folder called "**test**" that mirrors the package structure of your classes in the standard src folder. This allows the test code to be separated but ensures they are within the same namespace and share accessibility. Open the "**CounterTest**" class, which is actually a JUnit Test Case, and study the code and comments. Then run the program - all tests should pass. You may wish to purposely make a test fail to see the result of doing this.

Notes - Implementing your own classes:

- You can assume correct input for method parameters.
- The range of values expected for arguments can be stated in the Javadoc comments before each method (see explanation of Javadoc in the separate Javadoc notes).
- If someone using your class uses a method incorrectly, when it is specified how it should be used (e.g. what values a method accepts) then it is the user's fault.
- Therefore, do not do unnecessary error checking in the class, this should be done in the client application using the class.

Notes - Demonstrating and Testing your own classes:

- A main method application can be used to demonstrate or informally test the general behaviour of your class. To do so, create another class with a main() method, e.g. CounterDemo.
- If an application is being used to demonstrate the functionality of a specific class, then you can use 'App' or 'Demo' as a suffix, e.g. CounterApp or CounterDemo. However, you may wish to give it a more purposeful name if it is an application that uses a variety of classes, and is covering a more general range of use cases.
- To more rigorously test a class, you can use the JUnit testing framework, packaged with Eclipse.
- A JUnit Test Case can be created, e.g. CounterTest, to write unit tests for your class.
- Typically each constructor and method is tested one or more times in individual units forming an overall test case.
- A range of arguments and boundary values should be used to exercise applicable methods.

Specifying classes with UML notation

Before implementing a class we give a specification and documentation. To include:

- A paragraph (or more) to state the object's purpose, its salient properties (i.e. main attributes representing state), and its characteristic behaviour (i.e. methods).
- A list of the attributes (i.e. fields), and any constraints on their values (i.e. data type).
- A list of the methods in the public interface, and a brief description of their purpose. Any constraints on parameters, pre-conditions or particular behaviour should be noted.

A summary of a class template can be given in a UML class box, e.g.

OrderLine	Class name
- id : String - unitPrice : int //pence - quantity : int	attributes (i.e. fields)
+ getCost() : int + setQuantity(int) :void	some methods

Not all methods are necessarily listed in the UML class diagram; just those of particular interest. However, a class implementer will have to complete the design and identify all constructors and methods. Typically, all classes should have a **default** and **custom constructor**, **'get'** and **'set'** methods for each of the fields and a **toString()** method to output a textual representation of the object's current state. These are therefore not always included in the initial UML specification:

Question 2

Continue working in the same project and view the "**Name**" class within the src > lib package.

- a) Study the code and in particular look at how the UML specification below has been implemented. There are 'set' methods and a toString(), even though these are not in the UML class box, which often merely acts as a guide at an early design stage. There is no main method in the class as the Name class represents a new custom type that we are introducing into the language, in the same way the Counter class did.

Name
- firstName : String - familyName : String
+ getFirstName() : String + getFamilyName() : String + getFullName() : String

A Name class stores the first name and a family name of a person.

getFullName() combines the firstName and familyName.

- b) In the src > main package you will also see the "**NameDemo**" program (with a main method), which demonstrates some of the functionality of the Name class and also shows an example of an array of custom Name objects.
- c) Try temporarily removing the toString() method from the Name class, and then run the

demo program again (the output will have changed) - can you see that you still gain access to a toString method? Every class you create automatically inherits some methods from a class called Object and toString is one of these, however, it is standard practice to override it with your own implementation.

- d) Study the unit tests within the "**NameTest**" program in the test > lib package. There is not much to actually test in this case as Name is a relatively simple class.

Question 3 Now it's over to you; within your current project, you will note that within the test > lib package there is a "**CDTrackTest**" JUnit Test Case containing unit tests for each of the expected constructors and methods of a CDTrack class (yet to be developed). If you run them, currently all of them will fail.

Using these tests and the UML class box below (noting the three required private fields, as well as the public methods), develop a new class representing a "**CDTrack**" within the src > lib package (noting you will not require a main method in this class). You should periodically run the corresponding unit tests until hopefully all of them pass! You are essentially engaging in test-driven development.

You should now create a simple demonstration program called "**CDTrackApp**" with a main method inside the src > main package. As an example you could create one CDTrack object using the default constructor and then use the set methods to assign meaningful details, then create another CDTrack object using the custom constructor. Output (using System.out.println) the details of the first CDTrack object using the get methods, and then use the toString method for outputting the second.

CDTrack
- title : String - artist : String - duration : int
...get methods... ...set methods... + toString() : String

A CD Track has a title, artist and duration. The duration is stored as seconds and should be a non-negative number at all times.

Question 4 (*optional challenge - please attempt all other questions first*) Within your current Java project, inside the src > lib package create a new class called "**ModuloCounter**", note that you do not need a main method in a custom class as it is not a test program, and then:

- Implement a class similar to Counter, but which has some more functionality. Specifically, as well as storing the count value it should store a **max** value in an appropriate field. Write constructors and methods, but for the increment and decrement method, you need to ensure that the count value is always within the range **0..max-1** inclusive. If it is incremented (or decremented) beyond these limits it should 'wrap' round. E.g. a modulo 5 counter increments as follows: 0 1 2 3 4 0 1 2 3 4 0 1 2 3 ...
- In the src > main package, create a new Java class called "**ModuloCounterDemo**" and then within the main method write some code to assess if your new class works, specifically focusing on the wrapping functionality.

- c) Modify the ModuloCounter so that it has an additional field of type int called **cycles**, to keep count of how many times the counter has wrapped round, then add some more test code to your testing class.
- d) Question: could your **cycles** field have a data type that is of type Counter, instead of int? You may discuss this with your tutor and if you feel ambitious try to implement the change - it isn't that difficult but it shows a classic composition relationship between two classes, which you will explore more later in the module.
- e) If you're feeling adventurous, you could have a go at writing a JUnit Test Case (with a variety of appropriate unit tests) for the ModuloCounter class called "**ModuloCounterTest**" within the test > lib package.

Notes – Local variable type inference:

- The var identifier may optionally be used for type inference of local variables e.g. those declared in a main method.
- The variable must be initialised on the same line, and this technique cannot be used for field declarations or method parameters.
- The use of [var](#) comes down to personal preference and context. In our examples we sometimes use it to omit repetitiveness where the type information is clear from the initialisation. However, in some cases using it can make code less readable.

Please turn over...

Overriding the equals() method

A fundamental part of Object Orientated programming is the concept of inheritance. This is where one class inherits from another, i.e. a *subclass* is a specialized version of a *superclass*. There is one class in Java that every other class automatically inherits from (i.e. extends) - the Object class.

The Object class defines some very general methods including toString() and equals(...). Although any custom Class you create will automatically inherit these, you often want to *override* them to make your own implementations.

If you use the == operator to compare objects it will compare their object references (i.e. memory addresses). The same applies if you use the default equals(...) method inherited from the Object class. Therefore to compare objects created from your own custom classes you need to override .equals(...) to ensure it compares the internal contents (i.e. state) of the two objects, rather than their references.

Associated Reading:

Java Oracle Tutorials,

<https://docs.oracle.com/javase/tutorial/java/IandI/objectclass.html>

Question 5 Create a new Java project called "**equals**" (you do not need a module-info file) and then within it create two packages called "**lib**" and "**main**". Copy the **Name** class from your other project into the lib package, then within the main package create a new demo program called "**EqualsDemo**" and within the main method do the following:

- a) Copy the code below (displaying an equality test of two strings). We are purposely not using String Literals as they behave differently when testing for equality (as you may remember from when strings were first introduced). Before you run the program guess what the output will be and think about why that is:

```
String a = new String("hello");
String b = new String("hello");
System.out.println("a==b is: " + (a==b));
System.out.println("a.equals(b) is: " + a.equals(b));
```

- b) Now below your existing code in the main method copy the following code. Again, try and guess what the output will be before you run the program - were the results what you expected and can you explain why?

```
Name n = new Name("Joe", "Bloggs");
Name n1 = new Name("Joe", "Bloggs");
System.out.println("n==n1 is: " + (n==n1));
System.out.println("n.equals(n1) is: " + n.equals(n1));
```

Please turn over...

Question 6 You are now going to write your own equals(...) method implementation by overriding that which is inherited from the Object class. Copy the following code into your "Name" class, then run the code in your "EqualsDemo" program again and observe the results.

```
@Override
public boolean equals(Object obj) {
    // compare first & family names using String equals() method if
    obj passed in is instance of Name class.
    return (obj instanceof Name other)
        && this.familyName.equals(other.familyName)
        && this.firstName.equals(other.firstName);
}
```

Notes - Inheriting equals(...) from the Object superclass:

- You may still be wondering why the String and Name classes behaved as they did when getting tested for equality. String objects compared successfully whereas Name objects did not. Additionally, how were you able to invoke the equals(...) method on a Name object when the Name class does not contain an implementation of it?
- Every class in Java automatically extends the Object class and therefore inherits its methods and one of these is equals(...), therefore if you do not write your own implementation and override that specified in the superclass, you will be using the inherited one.
- The String class has its own version of equals(...) that compares the contents of a string, whereas the version inherited by Name would simply compare object references, acting similarly to the == equality operator.

Notes - Overriding equals(...) in your own classes:

- When you override a method specified in a superclass you firstly need to ensure the method header is identical. In this case, the return type should be **boolean**, the name equals and a single parameter of type Object.
- You then need to test some exceptional cases (that do happen); that the argument object passed in is not null, to avoid a NullPointerException, and that the class type is compatible to avoid a potential ClassCastException. The **instanceof** operator conducts both these checks. Pattern matching then downcasts the parameter "obj" (from being stored in a variable of type Object) to a Name variable.
- Finally, you want to ensure all instance variables (i.e. fields) of both objects share the same values. Typically the standard way to do this is use the == operator for primitive types and the equals() method for reference types.
- You may have noticed the **@Override** annotation placed above your method - this is not compulsory and it does not change how your code behaves, although it ensures that you, and more importantly the compiler, are aware that your method is intending to override another method. Should you accidentally name your method incorrectly or provide a different parameter type then your program will not compile.